CASO DE ESTUDIO 1: ENFOQUE ÁGIL

Proyecto: PilarBus — Ecosistema Móvil de Transporte Público

Informe de metodología ágil, estimación, tablero Kanban y control de versiones

1. Contexto y objetivo

PilarBus es una aplicación móvil y plataforma web para que los ciudadanos consulten recorridos de colectivos en tiempo real. El proyecto presenta incertidumbre: las funciones pueden cambiar por los comentarios semanales de usuarios beta, y el equipo debe integrar código continuamente sin interrumpir el servicio. Por eso se necesita una forma de trabajo adaptable, colaborativa y con entregas frecuentes.

2. Justificación metodológica: Scrum y flujo continuo

Se recomienda Scrum para planificar y revisar el trabajo en ciclos cortos (sprints), combinado con prácticas de flujo continuo visualizadas mediante Kanban. Scrum permite priorizar el Product Backlog, elegir un objetivo de sprint, revisar un incremento funcional y adaptar el plan con base en la retroalimentación de usuarios beta.

Beneficios frente a un enfoque tradicional en cascada:

• Adaptación: los cambios en alertas, SUBE o reclamos pueden reordenarse en el backlog sin esperar al final del proyecto.

• Retroalimentación temprana: se prueban versiones parciales con usuarios y se corrigen problemas antes de que sean costosos.

• Transparencia: el tablero permite conocer el estado de cada tarea y detectar bloqueos.

• Entrega incremental: se priorizan funciones valiosas, por ejemplo consultar recorridos, antes de completar todas las funciones.

• Calidad: las revisiones de código y las pruebas automáticas reducen el riesgo al integrar cambios.

El enfoque tradicional puede servir cuando los requisitos son estables, pero en este caso sería menos flexible porque los cambios semanales podrían generar retrabajo y demorar la entrega de valor.

3. Estimación ágil de esfuerzo y costos

Se usan Story Points (puntos de historia) para estimar complejidad, esfuerzo, incertidumbre y riesgo de manera relativa; no equivalen a horas exactas. La escala propuesta es 2, 5 y 8 puntos: 2 = sencillo y conocido; 5 = complejidad media; 8 = alta complejidad o incertidumbre. El equipo puede estimar en una sesión de Planning Poker y acordar el valor final.

Estimación inicial de las funcionalidades

Tarea	Puntos	Rol principal	Prioridad
Configurar repositorio público en GitHub	2	Desarrollador	Alta
Diseñar interfaz móvil y web	5	Diseño	Alta
Integrar mapa y paradas de colectivos	8	Desarrollador	Alta
Investigar e integrar datos de SUBE	8	Desarrollador	Media
Crear formulario de reclamos	5	Desarrollador	Media
Mostrar ubicación de colectivos en tiempo real	8	Desarrollador	Alta
Implementar alertas de desvíos	5	Desarrollador	Alta

Tarea	Puntos	Rol principal	Prioridad
Configurar integración continua y pruebas (CI/CD)	5	Desarrollador / Tester	Alta

Cómo se estiman costos: primero se estima el esfuerzo relativo con Story Points. Luego, cuando el equipo conoce su velocidad (puntos completados por sprint), se calcula cuántos sprints podría requerir el alcance priorizado. El costo monetario se obtiene multiplicando el tiempo real estimado por la tarifa del equipo y sumando infraestructura/servicios. No se asigna un precio inventado porque no se informaron tarifas, duración del sprint ni velocidad histórica.

La estimación es dinámica: después de cada sprint se comparan estimaciones y resultados, se revisan supuestos y se reestima el backlog si cambian los requisitos. Las funciones con dependencias externas, como SUBE y datos GPS, tienen más incertidumbre y deben investigarse temprano.

4. Gestión visual: tablero Kanban en Trello

Tablero: **PilarBus — Desarrollo Ágil**. Listas: To Do (Por hacer), In Progress (En progreso) y Done (Hecho). Las tarjetas contienen descripción, Story Points, rol responsable, prioridad y checklist. En el archivo CSV incluido se encuentran las ocho tarjetas iniciales.

Regla WIP (Work In Progress)

El límite de la columna In Progress es de **3 tarjetas como máximo**. Si ya hay tres tareas en progreso, el equipo no inicia otra: colabora para terminar una, resolver bloqueos o revisar el trabajo existente. Esto reduce el cambio constante de contexto, hace visibles los cuellos de botella y favorece terminar antes de empezar.

Etiquetas sugeridas

• Azul: Desarrollador

• Verde: Tester

• Amarillo: Diseño

• Rojo: Prioridad alta

• Violeta: CI/CD

Tarjetas iniciales y estados propuestos

Por hacer (5): repositorio público; interfaz móvil/web; mapa y paradas; investigación/integración SUBE; formulario de reclamos. En progreso (3): ubicación en tiempo real; alertas de desvíos; integración continua y pruebas. Hecho: inicialmente vacío; solo se mueve una tarjeta a Done cuando el trabajo esté implementado, probado y cumpla los criterios de aceptación. Los estados son una propuesta inicial de planificación, no evidencia de tareas ya realizadas.

5. Control de versiones y CI/CD con GitHub

Se propone un repositorio público llamado **PilarBus** con README.md, .gitignore y un workflow de GitHub Actions. El archivo de workflow incluido ejecuta una validación básica al abrir o actualizar un Pull Request y al hacer push a main. Como no se proporcionó código fuente ni tecnología concreta, el workflow deja indicado un paso de validación adaptable; debe reemplazarse por los comandos reales de instalación, lint y pruebas del proyecto.

Flujo de Pull Request

• Crear una rama para cada tarea y realizar cambios pequeños.

• Abrir un Pull Request hacia main y describir el cambio y las pruebas realizadas.

• GitHub Actions ejecuta las validaciones configuradas (Build/Test).

• El equipo revisa el código y corrige fallos; no se integra si las comprobaciones fallan.

• Tras la aprobación y las pruebas, se integra el cambio. El despliegue a producción debe configurarse por separado según el hosting y el entorno real.

CI significa Integración Continua: integrar cambios frecuentemente y validarlos automáticamente. CD puede significar Entrega Continua (el artefacto queda listo para publicar) o Despliegue Continuo (se publica automáticamente). Para PilarBus, primero se recomienda automatizar build y tests; el despliegue requiere definir hosting, secretos, permisos y estrategia de reversión.

6. Conclusión

Scrum aporta planificación iterativa y revisión frecuente; Kanban aporta visibilidad del flujo y control del trabajo simultáneo. La estimación relativa permite priorizar y actualizar previsiones sin fingir una precisión que aún no existe. GitHub, Pull Requests y CI/CD ayudan a mantener la calidad al integrar cambios. Esta combinación se ajusta a PilarBus porque el producto debe evolucionar con el feedback de usuarios y seguir siendo confiable.